

STUDY WITH ME

Azure Data Fundamentals

Microsoft Certified

By Nicolý Oliveira

CONTEÚDOS ABORDADOS

1

Descrever maneiras de representar dados

-  Descrever recursos de dados estruturados
-  Descrever características de dados semiestruturado
-  Descrever recursos de dados não estruturados

2

Identificar opções para armazenamento de dados

-  Descrever formatos comuns para arquivos de dados
-  Descrever tipos de bancos de dados

3

Descrever cargas de trabalho de dados comuns

-  Descrever recursos de cargas de trabalho transacionais
-  Descrever recursos de cargas de trabalho analíticas

4

Identificar funções e responsabilidades para cargas de trabalho de dados

-  Descrever as responsabilidades dos administradores de banco de dados
-  Descrever as responsabilidades dos engenheiros de dados
-  Descrever as responsabilidades dos analistas de dados

01.

Descrever maneiras de representar dados

Descrevendo formato dos dados...

Dados Estruturados



Definição:

São dados altamente organizados, armazenados em um formato fixo e padronizado, geralmente dentro de banco de dados relacionais que utilizam a linguagem SQL.



Características:

- Seguem um esquema rígido (colunas, linhas, tabelas).
- Fáceis de pesquisar e consultar usando SQL.
- Normalmente armazenados em bancos de dados relacionais (RDBMS).



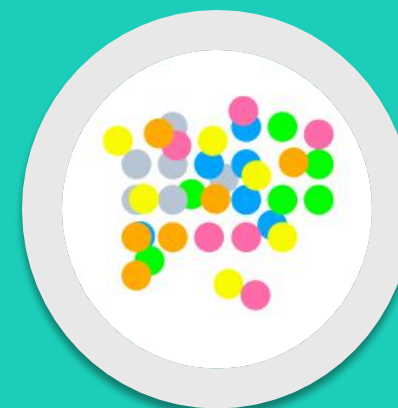
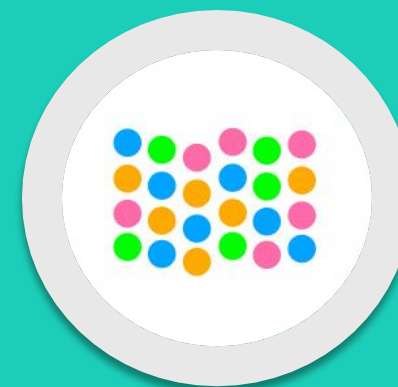
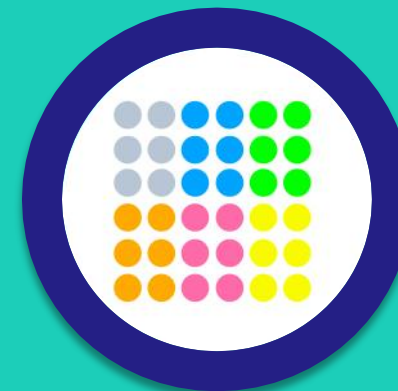
Exemplos:

- Tabelas de um banco de dados (clientes, pedidos, produtos).
- Planilhas do Excel com colunas bem definidas.
- Dados de sensores organizados em tabelas.



Onde é usado?

Aplicações comerciais, ERPs, CRMs, bancos de dados de transações financeiras.



Descrevendo formato dos dados...

Dados Semi - Estruturados



Definição:

São dados que não seguem um esquema rígido como os estruturados, mas ainda possuem alguma organização através de metadados (tags, atributos, chaves).



Características:

- Não seguem um modelo fixo de tabelas e colunas, mas ainda contêm elementos estruturais.
- Usam formatos flexíveis como JSON, XML e YAML.
- Podem ser armazenados em bancos NoSQL (MongoDB, Firebase, Cassandra)



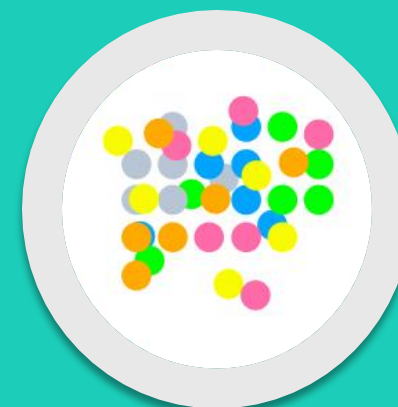
Exemplos:

- JSON e XML usados em APIs e aplicações web.
- Arquivos de logs do sistema.
- E-mails (estruturados pelo remetente, destinatário, assunto, mas com um corpo de texto livre).



Onde é usado?

Aplicações modernas, big data, APIs, sistemas que precisam de flexibilidade.



Descrevendo formato dos dados...

Dados Não Estruturados



Definição:

São dados sem um formato predefinido, não organizados de forma padronizada, dificultando buscas e análises diretas



Características:

- Não seguem um modelo fixo de organização.
- Difíceis de armazenar e processar em bancos relacionais.
- Dependem de tecnologias como Data Lakes, Big Data e IA para análise.



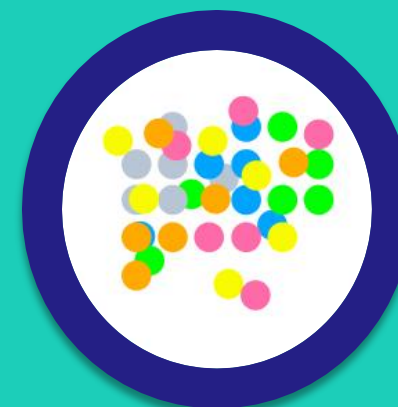
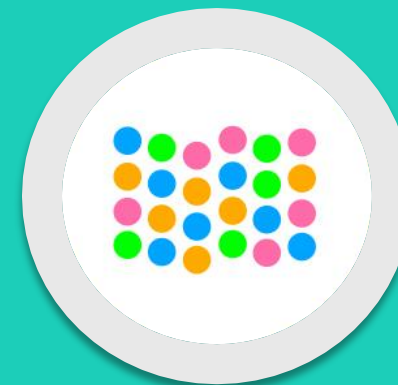
Exemplos:

- Arquivos de mídia (imagens, vídeos, áudios).
- Posts em redes sociais (textos livres, hashtags, emojis).
- Documentos em PDF e Word.



Onde é usado?

Processamento de imagens e vídeos, análise de sentimentos em redes sociais, NLP (Processamento de Linguagem Natural).



02. **Identificar opções para armazenamento de dados**

Formatos comuns para arquivos de dados

Os formatos de arquivos são padrões utilizados para armazenar e organizar informações de diferentes tipos de dados. Garantindo a compatibilidade e a integridade dos dados em diferentes sistemas e dispositivos.



Dados Estruturados

CSV, EXCEL, SQL, PARQUET, HDF5, AVRO, ORC ...

Dados Semi Estruturados

JSON, XML, YAML, BSON, PROTOBUF ...

Dados não estruturados

JPEG BMP, MP4, AVI, DOCX, PDF, MSG, TXT, ZIP, RAR, HTML, CSS, JS...

Formatos comuns para arquivos de dados – Estruturados

É comum questões referente aos seguintes tipos de dados, importante entender e memorizar o seu tipo, quando usado e estrutura.

Dados Estruturados - CSV

Formato : Texto simples, separado por virgula (ou ponto e vírgula, tab etc)

- Abre em Excel facilmente
- Cada linha é um registro
- Simples de ler e editar

```
id,nome,idade
1,Ana,25
2,Beto,30
3,Camila,22
```

Dados Estruturados - PARQUET

Formato : Binário, orientado a colunas.

- Não é legível diretamente em texto
- Visualização comum é feita por ferramentas como Apache Spark, Pandas, Athena,..

```
Schema:
{
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "nome", "type": "string"},
    {"name": "idade", "type": "int"}
  ]
}
```

```
Dados (armazenados por coluna):
id: [1, 2, 3]
nome: ["Ana", "Beto", "Camila"]
idade: [25, 30, 22]
```

Dados Estruturados - AVRO

Formato : Binário com esquema embutido

- Não legível diretamente, mas pode ser representado assim(em JSON com binário embutido)

```
// Schema separado
{
  "type": "record",
  "name": "Pessoa",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "nome", "type": "string"},
    {"name": "idade", "type": "int"}
  ]
}

// Dados serializados (binário)
[Binary data]
```

Dados Estruturados - ORC

Formato : Binário, orientado a colunas

- Não visível em texto puro
- Representação estrutural semelhante ao Parquet
- Suporta compressão, estatísticas e índices internos.

```
Schema:
struct<id:int, nome:string, idade:int>
```

```
Dados (armazenados por coluna):
id: [1, 2, 3]
nome: ["Ana", "Beto", "Camila"]
idade: [25, 30, 22]
```

Formatos comuns para arquivos de dados - Semi Estruturados

É comum questões referente aos seguintes tipos de dados, importante entender e memorizar seu tipo, quando usado e estrutura.

Semi Estruturados - JSON

Formato : Texto simples, baseado em objetos

- Troca de dados entre sistemas
- Estrutura hierárquica, com objetos e arrays
- Uso comum em REST APIs, mongoDB, configs, front/back-end

```
{
  "id": 1,
  "nome": "Ana",
  "idade": 25,
  "enderecos": [
    {
      "rua": "Rua A",
      "cidade": "SP"
    }
  ]
}
```

Semi Estruturados - XML

Formato : Texto, baseado em marcação (tags)

- Estrutura hierárquica, com tags aninhadas
- Uso comum em SOAP, Android, sistemas antigos, config de apps corporativas

```
< Pessoa >
  < id >1</ id >
  < nome >Ana</ nome >
  < idade >25</ idade >
  < enderecos >
    < endereco >
      < rua >Rua A</ rua >
      < cidade >SP</ cidade >
    </ endereco >
  </ enderecos >
</ Pessoa >
```

Semi Estruturados -YAML

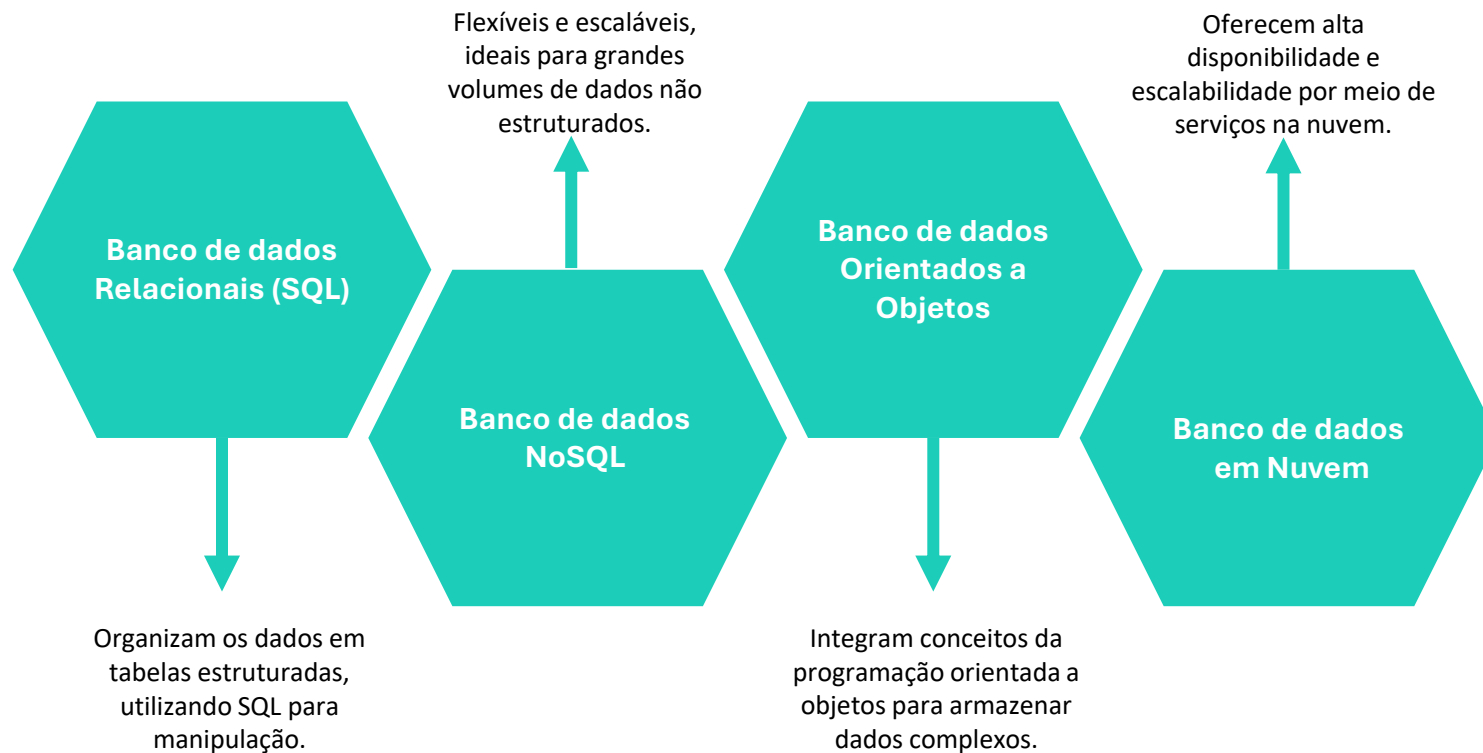
Formato : Texto leve e limpo, baseado em indentação

- Estrutura hierárquica, usa recuo (Indentação)
- Uso comum em Docker compose, Github actions, ansible, configs modernas.

```
id: 1
nome: Ana
idade: 25
enderecos:
  - rua: Rua A
    cidade: SP
```

Descrever tipos de banco de dados

Os bancos de dados são essenciais para armazenar e gerenciar informações de forma eficiente. Existem diferentes tipos de bancos de dados, cada um adequado para necessidades específicas.



Descrever tipos de banco de dados

Escalabilidade

Escalabilidade na computação em nuvem refere-se à **capacidade de expandir ou reduzir rapidamente os recursos** ou a potência de uma solução de TI, conforme a demanda. Embora o termo possa ser aplicado a qualquer sistema capaz de lidar com volumes crescentes de trabalho, ao falarmos em escalabilidade vertical ou horizontal, geralmente estamos nos referindo a bancos de dados.



A escalabilidade de bancos de dados é um dos principais pilares para o sucesso de aplicativos modernos. Imagine o lançamento de um novo app cuja base de usuários cresce rapidamente, de algumas dezenas para milhões em todo o mundo. Para lidar com esse aumento de demanda e garantir alta disponibilidade, é essencial que os desenvolvedores consigam escalar os recursos de forma eficiente, reduzindo o risco de falhas e minimizando o tempo de inatividade.

Vertical

- Aumenta ou reduz a capacidade de computação ou do banco de dados.
- Pode ser feita ajustando os níveis de desempenho (CPU, RAM, etc.).
- Utiliza pools de banco de dados elástico para adaptação automática à carga.
- Simples de implementar, mas tem limite físico de escalabilidade.

Horizontal

- Adiciona mais bancos de dados ou divide um banco grande em nós menores.
- Usa fragmentação (sharding) para distribuir dados entre servidores.
- Ideal para aplicações com grande volume de acesso simultâneo.
- Requer um gerenciamento mais complexo, mas oferece alta escalabilidade.

Descrever tipos de banco de dados

Quando escalar verticalmente?

A escala vertical é usada quando você precisa reagir rapidamente para consertar um problema de desempenho que não pode ser resolvido usando técnicas clássicas de otimização do banco de dados, como alterações de consulta ou indexação. Escalar verticalmente é útil para lidar com picos nas suas cargas de trabalho em que o nível de desempenho atual não pode atender a todas as demandas.



Observar que as suas cargas de trabalho estão atingindo algum limite de desempenho, como limites de CPU ou de E/S.



Precisar reagir rapidamente para consertar problemas de desempenho que não podem ser resolvidos com a otimização do banco de dados clássica.



Precisar de uma solução que permita alterar as camadas de serviço para se adaptar à alteração dos requisitos de latência.

Descrever tipos de banco de dados

Quando escalar horizontalmente?

Os desenvolvedores de aplicativos começarão a considerar escalar horizontalmente ou expandir quando não conseguirem obter recursos suficientes para as cargas de trabalho, mesmo operando nos níveis de desempenho mais altos. Com a escala horizontal, os dados são divididos em vários bancos de dados (ou fragmentos) entre servidores, e cada fragmento pode ser escalado ou reduzido verticalmente de modo independente.



Você tiver aplicativos distribuídos geograficamente em que cada aplicativo deve acessar parte dos dados na região. Cada aplicativo acessará apenas o fragmento associado a essa região sem afetar outros fragmentos.



Você tiver um cenário de fragmentação global, como balanceamento de carga, em que há um grande número de clientes distribuídos geograficamente que inserem dados nos próprios fragmentos dedicados.



Você atingir o limite dos seus requisitos de desempenho, mesmo nos níveis de desempenho mais altos do seu serviço, ou se os dados não couberem em um banco de dados individual.

Descrever tipos de banco de dados

Latência

Latência é o tempo entre o momento em que uma requisição (consulta, inserção, atualização etc.) é feita ao banco de dados e o momento em que a primeira resposta começa a ser retornada ao cliente.



Ela é um indicador de desempenho crítico, principalmente em sistemas com requisitos de tempo real ou alta concorrência.

Baixa

- Tempo de resposta baixo e consistente (ex: < 10ms em bancos transacionais, ou < 100ms em analíticos).
- Indicativo de que o sistema é bem projetado e escalado para o tipo de workload.

Exemplos

- Sistemas OLTP (transacionais) como bancos, e-commerces, ERPs.
Ex: um banco de dados PostgreSQL respondendo a uma consulta de saldo em 3ms.
- Sistemas em tempo real (IoT, jogos, stream de dados):
Ex: Redis sendo usado para leitura de cache com latência de 1ms.

Alta

- Tempo de resposta acima do aceitável para o caso de uso.
- Pode ser sintoma de gargalo em rede, disco, CPU ou de modelo de dados mal projetado.

Exemplos

- Uma consulta em um data lake com arquivos Parquet mal particionados, levando 20 segundos para retornar.
- Um JOIN pesado entre tabelas grandes sem índices ou filtros, demorando 2 minutos.
- Um banco relacional tentando responder a uma consulta analítica sem estar otimizado

Descrever tipos de banco de dados

Tipos de Armazenamento – Relacional

Fornece um dialeto do SQL para recuperar e gerenciar dados. Um RDBMS normalmente implementa um mecanismo transaccionalmente consistente que esta em conformidade com o modelo ACID.

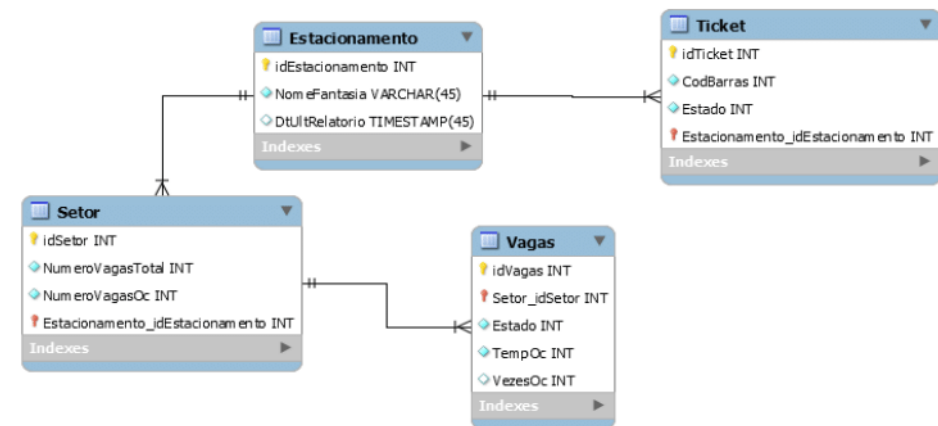
Um RDBMS normalmente dá suporte a um modelo de esquema em gravação, em que a estrutura de dados é definida com antecedência e todas as operações de leitura ou gravação devem usar o esquema. Os dados em um RDBMS devem ser normalizados, o que não é apropriado para cada conjunto de dados

Carga

- Os registros são frequentemente criados e atualizados
- Várias operações devem ser concluídas em uma única transação
- As relações são impostas usando restrições de banco de dados
- Os índices são usados para otimizar o desempenho da consulta

Exemplo

- Gerenciamento de Inventario
- Gerenciamento de Pedidos
- Banco de dados de Relatórios
- Contabilidade



Descrever tipos de banco de dados

Tipos de Armazenamento – Chave/Valor

Um repositório de chave/valor associa cada valor de dados a uma chave exclusiva.

A maioria dos repositórios assim da suporte apenas a operações simples de consulta, inserção e exclusão. Para modificar um valor (parcial ou completamente), um aplicativo deve substituir os dados existentes para todo o valor. Na maioria das implementações, ler ou gravar um único valor é uma operação atômica.

Os repositórios de chave/valor são altamente otimizados para aplicativos que executam pesquisas simples, mas são menos adequados se você precisar consultar dados em diferentes repositórios de chave/valor. Os repositórios de chave/valor também não são otimizados para consulta por valor

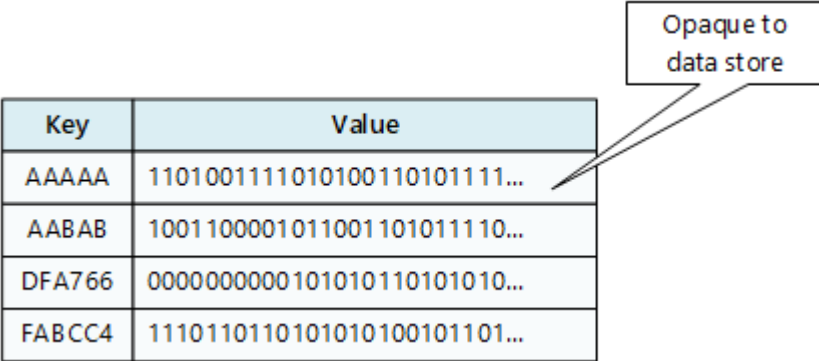
Carga

- Os dados são acessados usando uma única chave, como um dicionário.
- Associações, bloqueios ou uniões não são necessários.
- Nenhum mecanismo de agregação é usado.
- Índices secundários geralmente não são usados.

Exemplo

- Cache de dados
- Gerenciamento de sessão
- Preferência do usuário e gerenciamento de perfil
- Recomendação do produto e serviço de anúncios

Key	Value
AAAAA	11010011111010100110101111...
AABAB	1001100001011001101011110...
DFA766	0000000000101010110101010...
FABCC4	1110110110101010100101101...



A diagram showing a table with two columns: 'Key' and 'Value'. The table contains four rows of data. A callout box with the text 'Opaque to data store' has two lines pointing to the 'Value' column of the first two rows, indicating that the values are opaque to the data store.

Descrever tipos de banco de dados

Tipos de Armazenamento - Documentos

Um banco de dados de documentos armazena uma coleção de documentos , em que cada documento consiste em campos e dados nomeados.

Os dados podem ser valores simples ou elementos complexos, como listas e coleções filho. Os documentos são recuperados por chaves exclusivas.

Normalmente, um documento contém os dados de uma única entidade, como um cliente ou um pedido.

Um documento pode conter informações que seriam distribuídas em várias tabelas relacionais em um RDBMS.

Os documentos não precisam ter a mesma estrutura. Os aplicativos podem armazenar dados diferentes em documentos à medida que os requisitos de negócios são alterados.

.

Carga

- As operações de inserção e atualização são comuns.
- Não há incompatibilidade de impedância objeto-relacional. Os documentos podem corresponder melhor às estruturas de objeto usadas no código do aplicativo.
- Documentos individuais são recuperados e gravados como um único bloco.
- Os dados exigem índice em vários campos.

Exemplo

- Catálogo de produtos
- Gerenciamento de conteúdo
- Gerenciamento de inventário

Key	Document
1001	{ "CustomerID": 99, "OrderItems": [{ "ProductID": 2010, "Quantity": 2, "Cost": 520 }, { "ProductID": 4365, "Quantity": 1, "Cost": 18 }], "OrderDate": "04/01/2017" }
1002	{ "CustomerID": 220, "OrderItems": [{ "ProductID": 1285, "Quantity": 1, "Cost": 120 }], "OrderDate": "05/08/2017" }

Descrever tipos de banco de dados

Tipos de Armazenamento - Graph

Um banco de dados de grafo armazena dois tipos de informações, nós e bordas. As bordas especificam os relacionamentos entre os nós.

Nódulos e arestas podem ter propriedades que oferecem informações sobre aquele nódulo ou aresta, semelhantes às colunas em uma tabela.

As bordas também podem ter uma direção que indica a natureza da relação.

Os bancos de dados do Graph podem executar consultas com eficiência na rede de nós e bordas e analisar as relações entre entidades. Normalmente, um documento contém os dados de uma única entidade, como um cliente ou um pedido. Um documento pode conter informações que seriam distribuídas em várias tabelas relacionais em um RDBMS.

Os documentos não precisam ter a mesma estrutura. Os aplicativos podem armazenar dados diferentes em documentos à medida que os requisitos de negócios são alterados.

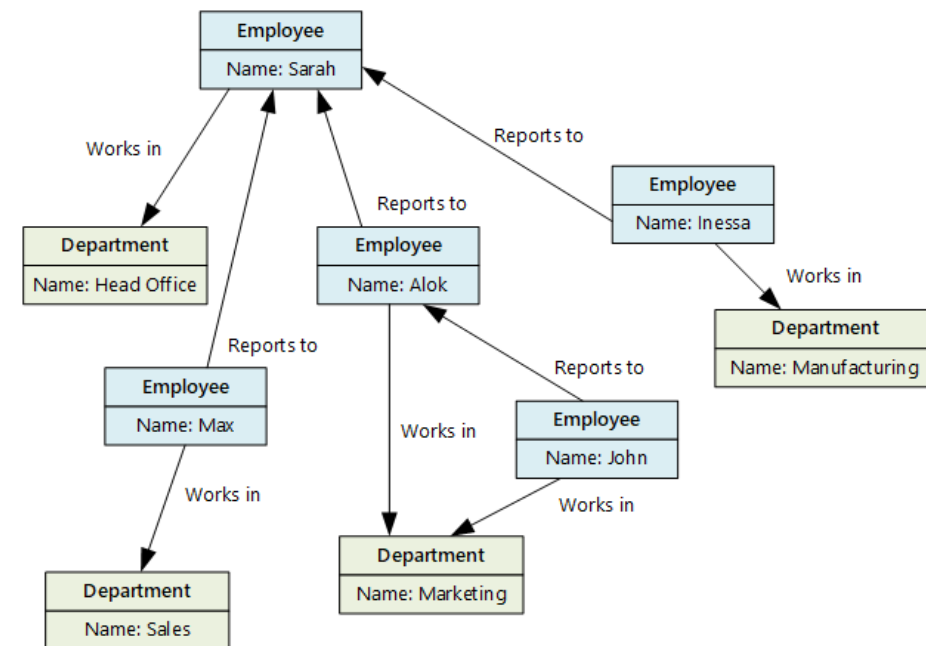
.

Carga

- Relações complexas entre itens de dados envolvendo muitos saltos entre itens de dados relacionados.
- A relação entre itens de dados é dinâmica e alterada ao longo do tempo.
- As relações entre objetos são de primeira classe, sem precisar de chaves estrangeiras e ingressos para serem percorridas.

Exemplo

- Gráficos da organização
- Grafos sociais
- Detecção de fraude
- Mecanismos de recomendação



Descrever tipos de banco de dados

Tipos de Armazenamento – Análise de Dados

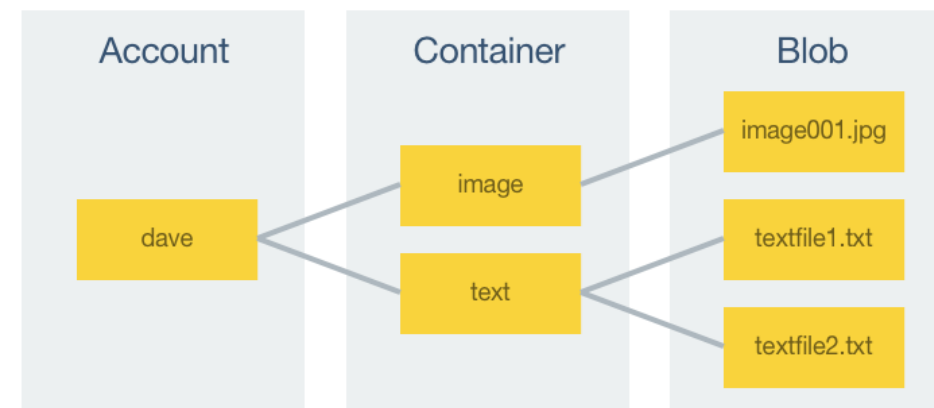
Os repositórios de análise de dados fornecem soluções massivamente paralelas para ingestão, armazenamento e análise de dados. Os dados são distribuídos entre vários servidores para maximizar a escalabilidade. Formatos de arquivo de dados grandes, como CSV (arquivos delimitadores), parquet ORC são amplamente usados na análise de dados. Dados históricos normalmente são armazenados em armazenamentos de dados, como o Armazenamento de Blobs ou o Azure Data Lake Storage Gen2, que são acessados pelo Azure Synapse, pelo Databricks ou pelo HDInsight como tabelas externas. Um cenário típico que usa dados armazenados como arquivos Parquet para desempenho é descrito no artigo Usar tabelas externas com o SQL do Synapse.

Carga

- Análise de dados
- BI da empresa.

Exemplo

- Data Warehouse corporativo



Descrever tipos de banco de dados

Tipos de Armazenamento – Famílias de colunas

Um banco de dados da família de colunas organiza dados em linhas e colunas. Você pode pensar em um banco de dados baseado em famílias de colunas como armazenando dados tabulares com linhas e colunas, mas as colunas são divididas em grupos conhecidos como famílias de colunas. Cada família de colunas contém um conjunto de colunas que estão logicamente relacionadas e normalmente são recuperadas ou manipuladas como uma unidade. Outros dados acessados separadamente podem ser armazenados em famílias de colunas separadas. Dentro de uma família de colunas, novas colunas podem ser adicionadas dinamicamente e as linhas podem ser esparsas (ou seja, uma linha não precisa ter um valor para cada coluna).

Carga

- A maioria dos bancos de dados da família de colunas executa operações de gravação extremamente rapidamente.
- As operações de atualização e exclusão são raras.
- Projetado para fornecer alta taxa de transferência e acesso de baixa latência.
- Dá suporte ao acesso fácil de consulta a um determinado conjunto de campos dentro de um registro muito maior.
- Massivamente escalonável.

Exemplo

- Recomendações
- Personalização
- Dados do sensor
- Telemetria
- Mensagens, Análise de mídia social eAnálise da Web
- Monitoramento de atividades
- Clima e outros dados de série temporal

CustomerID	Column Family: Identity
001	First name: Mu Bae Last name: Min
002	First name: Francisco Last name: Vila Nova Suffix: Jr.
003	First name: Lena Last name: Adamczyk Title: Dr.

CustomerID	Column Family: Contact Info
001	Phone number: 555-0100 Email: someone@example.com
002	Email: vilanova@contoso.com
003	Phone number: 555-0120

Descrever tipos de banco de dados

Tipos de Armazenamento – Mecanismo de Pesquisa

Um banco de dados do mecanismo de pesquisa permite que os aplicativos pesquisem informações mantidas em armazenamentos de dados externos. Um banco de dados do mecanismo de pesquisa pode indexar grandes volumes de dados e fornecer acesso quase em tempo real a esses índices.

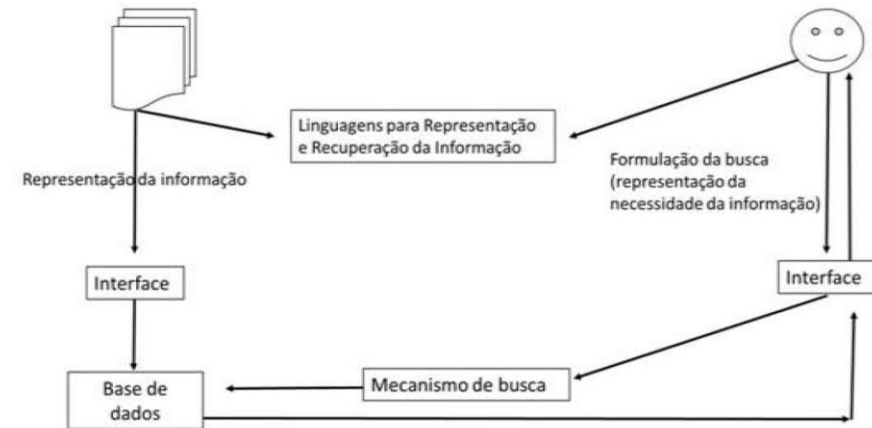
Os índices podem ser multidimensionais e podem dar suporte a pesquisas de texto livre em grandes volumes de dados de texto. A indexação pode ser executada usando um modelo de pull, disparado pelo banco de dados do mecanismo de pesquisa ou usando um modelo de push, iniciado pelo código do aplicativo externo.

Carga

- Índices de dados de várias fontes e serviços.
- As consultas são ad hoc e podem ser complexas.
- A pesquisa de texto completo é necessária.
- Requer consulta ad hoc de autoatendimento.

Exemplo

- Catálogos de produtos
- Pesquisa no site
- Registrar em log



Descrever tipos de banco de dados

Tipos de Armazenamento – Séries Temporais

Os dados de série temporal são um conjunto de valores organizados por tempo. Os bancos de dados de série temporal normalmente coletam grandes quantidades de dados em tempo real de um grande número de fontes. As atualizações são raras e as exclusões geralmente são feitas como operações em massa. Embora os registros gravados em um banco de dados de série temporal sejam geralmente pequenos, geralmente há um grande número de registros e o tamanho total dos dados pode crescer rapidamente.

Carga

- Os registros geralmente são acrescentados sequencialmente em ordem de tempo.
- Um proporção predominante das operações (95% a 99%) é composta por gravações.
- As atualizações são raras.
- As exclusões ocorrem em massa e são feitas em blocos ou registros contíguos.
- Os dados são lidos sequencialmente em ordem de tempo crescente ou decrescente, muitas vezes em paralelo.

Exemplo

- Monitoramento e telemetria de eventos.
- Sensor ou outros dados de IoT.

R

```
# Carregar pacotes
library(rbcdb)
library(purrr)
library(dplyr)

# Coleta de dados do Banco Central (SGS)
series_temporais <- get_series(
  code = c("dolar" = 3696, "ipca" = 433, "selic" = 4189),
  start_date = "2022-01-01"
) |>
  reduce(.f = full_join, by = "date")
series_temporais
```

date	dolar	ipca	selic
2022-01-01	5.3574	0.54	9.15
2022-02-01	5.1394	1.01	10.49
2022-03-01	4.7378	1.62	11.15
2022-04-01	4.9191	1.06	11.65
2022-05-01	4.7289	0.47	12.51
2022-06-01	5.2380	0.67	12.89
2022-07-01	5.1884	-0.68	13.15
2022-08-01	5.1790	-0.36	13.58
2022-09-01	5.4066	-0.29	13.65
2022-10-01	5.2570	0.59	13.65

Python

```
# Importar bibliotecas
from bcb import sgs

# Coleta de dados do Banco Central (SGS)
series_temporais = sgs.get(
    codes = {"dolar": 3696, "ipca": 433, "selic": 4189},
    start = "2022-01-01"
)
series_temporais
```

	dolar	ipca	selic
Date			
2022-01-01	5.3574	0.54	9.15
2022-02-01	5.1394	1.01	10.49
2022-03-01	4.7378	1.62	11.15
2022-04-01	4.9191	1.06	11.65
2022-05-01	4.7289	0.47	12.51
2022-06-01	5.2380	0.67	12.89
2022-07-01	5.1884	-0.68	13.15
2022-08-01	5.1790	-0.36	13.58
2022-09-01	5.4066	-0.29	13.65
2022-10-01	5.2570	0.59	13.65
2022-11-01	5.2941	0.41	13.65
2022-12-01	5.2177	0.62	13.65
2023-01-01	5.0993	0.53	13.65
2023-02-01	5.2078	0.84	13.65
2023-03-01	NaN	0.71	13.65
2023-04-01	NaN	NaN	13.65

Descrever tipos de banco de dados

Tipos de Armazenamento – Orientado a Objetos

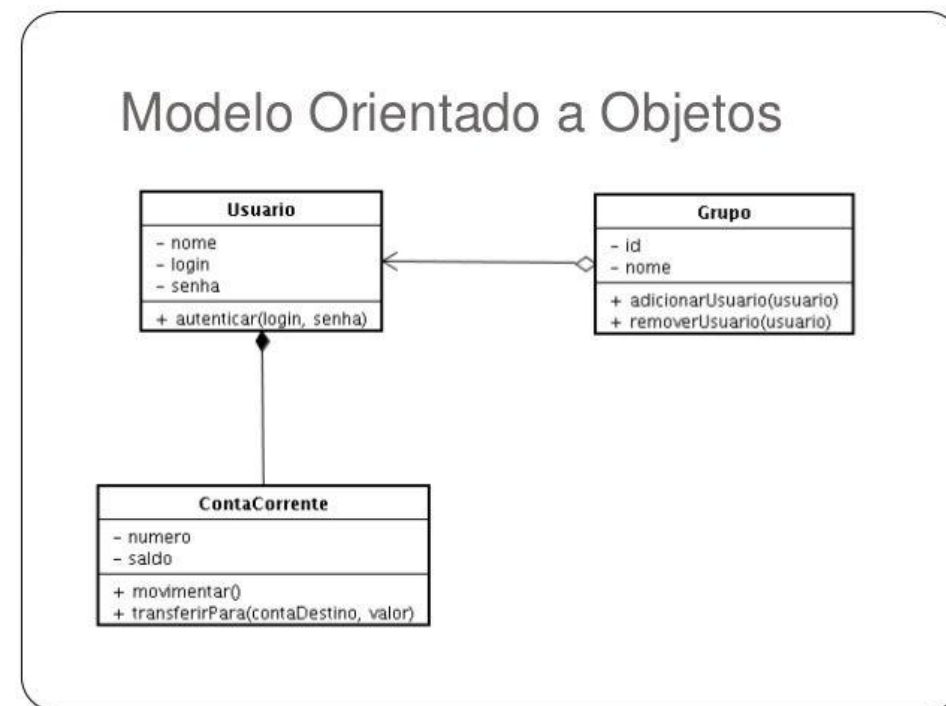
O armazenamento de objetos é otimizado para armazenar e recuperar grandes objetos binários (imagens, arquivos, fluxos de vídeo e áudio, documentos e objetos de dados de aplicativo grandes, imagens de disco de máquina virtual). Arquivos de dados grandes também são usados popularmente neste modelo, por exemplo, arquivo delimitado por vírgula (CSV), parquet ORC. Os repositórios de objetos podem gerenciar grandes quantidades de dados não estruturados.

Carga

- Identificado por chave.
- Normalmente, o conteúdo é um ativo, como um delimitador, uma imagem ou um arquivo de vídeo.
- O conteúdo deve ser durável e externo a qualquer camada de aplicativo.

Exemplo

- Imagens, vídeos, documentos do office, PDFs
- HTML estático, JSON, CSS
- Arquivos de log e auditoria
- Backups de banco de dados



Descrever tipos de banco de dados

Tipos de Armazenamento – Arquivos Compartilhados

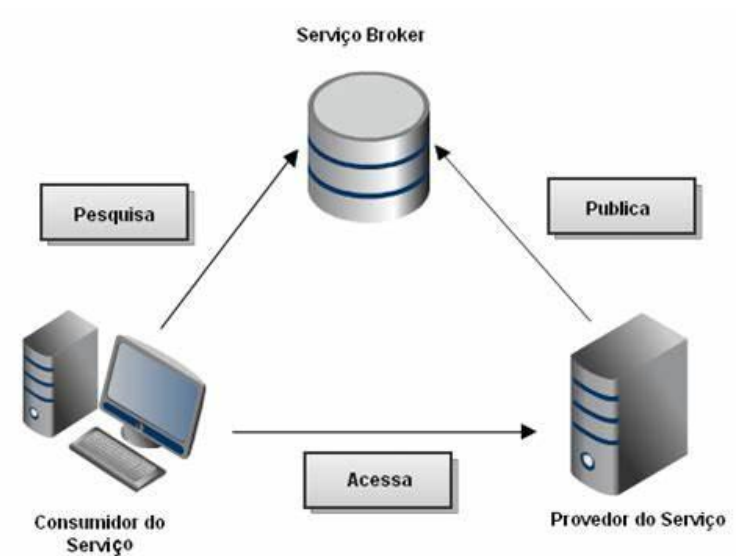
Às vezes, usar arquivos simples pode ser o meio mais eficaz de armazenar e recuperar informações. O uso de compartilhamentos de arquivos permite que os arquivos sejam acessados em uma rede. Dada a segurança apropriada e os mecanismos de controle de acesso simultâneos, o compartilhamento de dados dessa maneira pode permitir que os serviços distribuídos forneçam acesso a dados altamente escalonável para executar operações básicas e de baixo nível, como solicitações simples de leitura e gravação.

Carga

- Migração de aplicativos existentes que interagem com o sistema de arquivos.
- Requer a interface SMB.

Exemplo

- Arquivos herdados
- Conteúdo compartilhado acessível entre várias VMs ou instâncias de aplicativo



Descrever tipos de banco de dados

ACID

ACID é um acrônimo que representa quatro propriedades essenciais para garantir que as transações em um banco de dados sejam confiáveis. Isso é super importante pra manter a integridade dos dados, especialmente em sistemas críticos. Vamos destrinchar:

A

A transação é **atômica**, ou seja, ou **todas as operações** da transação são concluídas com sucesso, ou **nenhuma** delas é. Se algo der errado no meio do caminho, o banco **desfaz tudo** com um rollback. Nada de ficar com dados pela metade.

Atomicidade

C

A transação leva o banco de um estado válido para **outro estado igualmente válido**, respeitando todas as regras, restrições e integridade dos dados. Se você tem uma regra que não permite saldo negativo, por exemplo, isso deve ser mantido.

Consistência

I

Mesmo que várias transações aconteçam **ao mesmo tempo**, é como se cada uma estivesse sendo executada **sozinha**. O banco garante que o resultado final seja o mesmo de se elas tivessem sido feitas em sequência.

Isolamento

D

Depois que uma transação é confirmada (committed), os dados são **gravados de forma permanente** no banco, mesmo que role uma falha no sistema depois (como queda de energia).

Durabilidade

Descrever tipos de banco de dados

Os bancos de dados são essenciais para armazenar e gerenciar informações de forma eficiente. Existem diferentes tipos de bancos de dados, cada um adequado para necessidades específicas.

		 Azure Cosmos DB			
Nome	Azure SQL Database	Azure Cosmos DB	SQL Server	SQL Managed Instance	Azure Database for Postgree SQL
Modelo de Dados	Relacional (SQL)	NoSQL (Multi-modelo)	Relacional (SQL)	Relacional (SQL)	Relacional (SQL)
Escalabilidade	Vertical	Horizontal	Vertical	Vertical	Vertical
Compatibilidade	SQL Server	API para MongoDB, Gremlin, Cassandra, SQL, Table	Windows/Linux, On-premise e Azure	Totalmente compatível com SQL Server	PostgreSQL
Casos de Uso	Aplicações empresariais, ERP, CRM	Apps globais, IoT, Big Data	Aplicações transacionais e analíticas	Migração de cargas de trabalho SQL Server	Aplicações Open Source, GIS, Big Data
Tipo de Armazenamento	Blocos (Bancos Relacionais)	Documentos e Chaves/Valores	Blocos (Bancos Relacionais)	Blocos (Bancos Relacionais)	Blocos (Bancos Relacionais)

Descrever tipos de banco de dados

					
Nome	Azure Database MySQL	Azure Database for MariaDB	Azure Blob Storage	Azure Table Storage	Azure Cache for Redis
Modelo de Dados	Relacional (SQL)	Relacional (SQL)	Objeto	NoSQL (Chave-Valor)	NoSQL (Chave-Valor)
Escalabilidade	Vertical	Vertical	Horizontal	Horizontal	Horizontal
Compatibilidade	MySQL	MariaDB	SDKs para várias linguagens	API de Tabela	Redis
Casos de Uso	Aplicações Web, E-commerce	Aplicações Web e móveis	Armazenamento de arquivos, backups, mídia	Armazenamento de grandes volumes de dados não estruturados	Cache de alto desempenho, sessões de usuário
Tipo de Armazenamento	Blocos (Bancos Relacionais)	Blocos (Bancos Relacionais)	Blobs (Objetos)	Tabela distribuída	Memória

Descrever tipos de banco de dados

	 Azure Data Lake	 Azure Synapse Analytics			Azure Databricks 
Nome	Azure Data Lake Storage Gen2	Azure Syapse Analytics	Azure SQL Data Warehouse	Azure HDInsight	Azure Databricks
Modelo de Dados	Hadoop/Big Data	Relacional (SQL + Big Data)	Relacional (MPP)	Big Data (Hadoop, Spark, Kafka)	Big Data (Apache Spark)
Escalabilidade	Horizontal	Horizontal	Horizontal	Horizontal	Horizontal
Compatibilidade	HDFS, Azure Blob	SQL, Spark, Data Lake	SQL Server	Hadoop, Spark, HBase, Kafka	Spark, ML, AI
Casos de Uso	Análise de Big Data, Data Lakes	Data Warehousing e Analytics	Análise de dados estruturados	Big Data Analytics, ETL	Machine Learning, Processamento de dados
Tipo de Armazenamento	Arquivos distribuídos	Blocos e Arquivos distribuídos	Blocos (Bancos Relacionais)	Arquivos distribuídos	Arquivos distribuídos

03.

Descrever cargas de trabalho de dados comuns

Descrever recursos de cargas de trabalho transacionais e analíticas

OLAP E OLTP

Enquanto OLTP é voltado para o processamento eficiente de transações em tempo real, OLAP é focado na análise de grandes volumes de dados para geração de insights e apoio à decisão.



Entenda os dois pilares fundamentais dos sistemas de dados: OLAP e OLTP. Saber diferenciá-los é essencial para projetar soluções eficazes que atendam tanto à análise quanto às operações do dia a dia.

OLAP

Online Analytical Processing

Combina e agrupa os dados para que você possa analisá-los de diferentes pontos de vista

Objetivo Principal: Analisar dados agregados - Gerar relatórios, realizar análises complexas de dados e identificar tendências

Modelo de Dados: Dados multidimensionais para que você possa visualizar os mesmos dados de diferentes ângulos. Os bancos de dados OLAP armazenam dados em um formato de cubo, em que cada dimensão representa um atributo de dados diferente. Cada célula no cubo representa um valor ou medida para a interseção das dimensões.

Arquitetura de Dados: Leitura de dados sobre as operações de gravações de dados

OLTP

Online Transaction Processing

Armazena e atualiza dados transacionais de forma confiável e eficiente em grandes volumes.

Objetivo Principal: Processar as transações do banco de dados - Processar pedidos, atualizar estoque e gerenciar contas de clientes

Modelo de Dados: São unidimensionais e se concentram em um aspecto dos dados. Eles usam um banco de dados relacional para organizar os dados em tabelas. Cada linha na tabela representa uma instância da entidade e cada coluna representa um atributo da entidade.

Arquitetura de Dados: Prioriza as operações de gravação de dados

Descrever recursos de cargas de trabalho transacionais e analíticas

OLAP E OLTP

Enquanto OLTP é voltado para o processamento eficiente de transações em tempo real, OLAP é focado na análise de grandes volumes de dados para geração de insights e apoio à decisão.

OLAP Online Analytical Processing

Atributo	Descrição
Objetivo	Analisar grandes volumes de dados históricos para tomada de decisão
Operações comuns	Consultas complexas, agregações, drill-down, pivot
Dados acessados	Grandes volumes, muitos registros
Complexidade das consultas	Alta – com múltiplos joins e agregações
Frequência de acesso	Moderada – uso por analistas e gestores
Tempo de resposta esperado	Segundos a minutos, dependendo da complexidade e volume
Modelagem de dados	Desnormalização (modelos estrela ou floco de neve)
Atualização dos dados	Raramente – geralmente por cargas agendadas (batch)
Usuários típicos	Analistas de BI, gestores, áreas estratégicas

OLTP Online Transaction Processing

Atributo	Descrição
Objetivo	Registrar transações do dia a dia
Operações comuns	Inserção, atualização, deleção e consulta simples
Dados acessados	Pequenos volumes, registros individuais
Complexidade das consultas	Baixa – consultas simples, com poucos joins
Frequência de acesso	Alta – acessos simultâneos por vários usuários
Tempo de resposta esperado	Muito rápido (milissegundos a poucos segundos)
Modelagem de dados	Normalização alta (3ª forma normal) para reduzir redundância
Atualização dos dados	Frequente e em tempo real
Usuários típicos	Operacionais, administrativos, atendentes

Descrever recursos de cargas de trabalho transacionais e analíticas

DATAWAREHOUSE vs DATA LAKE

DATAWAREHOUSE

Um **Data Warehouse** é um repositório centralizado de dados estruturados, projetado para **análise e tomada de decisão**. Ele armazena informações provenientes de diversas fontes (ERPs, CRMs, bancos transacionais) de forma organizada e modelada (normalizada ou em estrela), otimizando consultas analíticas e relatórios gerenciais. Tecnologias como Snowflake, Amazon Redshift e Google BigQuery são exemplos modernos de data warehouses. Ele se destaca por aplicar **processos de ETL (Extract, Transform, Load)**, onde os dados são limpos, transformados e armazenados em um modelo rígido, com foco em consistência, integridade e desempenho. É ideal para **BI (Business Intelligence)**, pois permite análises consistentes sobre dados históricos e bem governados.

DATA LAKE

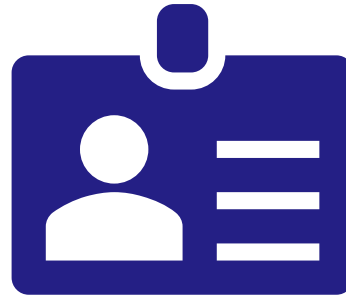
Um **Data Lake** é uma arquitetura de armazenamento que permite guardar grandes volumes de dados em seu formato bruto, sejam eles estruturados, semi-estruturados (como JSON, XML) ou não estruturados (como imagens, vídeos, logs). Ao contrário do Data Warehouse, no Data Lake os dados podem ser ingeridos rapidamente usando **ETL ou ELT**, e a estruturação ocorre apenas no momento da leitura ("schema-on-read"). Isso oferece flexibilidade e escalabilidade para análises exploratórias, ciência de dados e machine learning. Ele é comum em ambientes de Big Data e cloud, com ferramentas como Amazon S3, Azure Data Lake Storage ou Hadoop HDFS. Embora mais flexível, um Data Lake exige atenção com governança e qualidade de dados para não se tornar um "data swamp" (lago de dados inutilizável).

04

**Identificar funções e
responsabilidades para
cargas de trabalho de
dados**

Identificar funções e responsabilidades para cargas de trabalho de dados

Engenheiro de Dados (Data Engineer)



Projetar, construir e manter pipelines de dados de alto desempenho e confiáveis.



Integrar, transformar e consolidar dados de sistemas estruturados e não estruturados para soluções analíticas.



Garantir a segurança, eficiência e organização dos repositórios de dados, alinhando-se aos requisitos de negócios e restrições específicas.

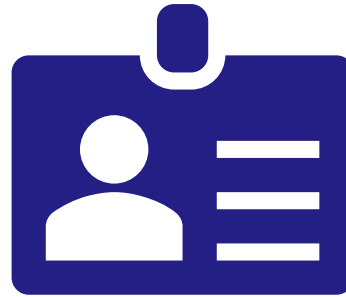


Trabalhar em estreita colaboração com engenheiros de análise, arquitetos, analistas e administradores para projetar e implantar soluções de engenharia de dados para análises.



Identificar funções e responsabilidades para cargas de trabalho de dados

Analista de Dados (Data Analyst)



Habilitar as empresas a maximizar o valor de seus ativos de dados por meio de ferramentas de visualização e relatórios.



Realizar o perfilamento, limpeza e transformação de dados.



Projetar e construir modelos de dados escaláveis e eficazes.

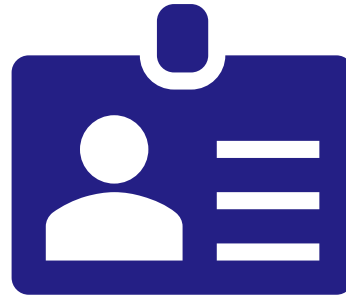


Implementar capacidades de análise avançada em relatórios para análise..



Identificar funções e responsabilidades para cargas de trabalho de dados

Cientista de Dados (Data Scientist)



Coletar, analisar e interpretar grandes volumes de dados para resolver problemas complexos de negócios



Combinar estatísticas, ciência da computação e conhecimento de negócios para ajudar a organização a entender mais sobre si mesma e alcançar seus objetivos.



Projetar e criar ambientes adequados para cargas de trabalho de ciência de dados.

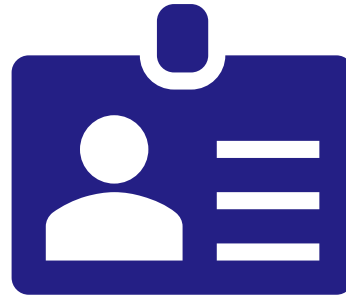


Explorar dados, treinar modelos de aprendizado de máquina e implementar pipelines para preparação de produção.



Identificar funções e responsabilidades para cargas de trabalho de dados

Administrador de Banco de Dados (DBA)



Implementar e gerenciar aspectos operacionais de soluções de plataforma de dados nativas da nuvem e híbridas.



Utilizar diversos métodos e ferramentas para realizar e automatizar operações diárias, incluindo o uso de Transact-SQL (T-SQL) para fins de gerenciamento administrativo.

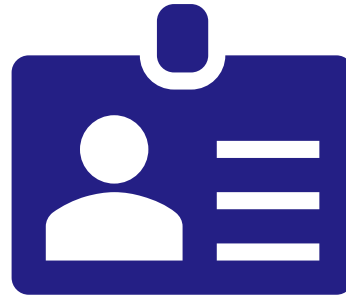


Gerenciar a configuração, manutenção, desempenho, segurança e recuperação de bancos de dados.



Identificar funções e responsabilidades para cargas de trabalho de dados

Arquiteto de Dados (Data Architect)



Traduzir requisitos de negócios em designs para soluções Azure que se alinhem ao Azure Well-Architected Framework e ao Cloud Adoption Framework for Azure..



Implementar soluções no Azure em parceria com desenvolvedores, administradores, engenheiros de segurança e engenheiros de dados



Garantir que as soluções sejam escaláveis, seguras e atendam aos objetivos de negócios..



05

Links Uteis